Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O enunciado pede a sincronização do embarque no metrô da METROREC: em C, com pthreads, definir a struct station e as funções station_init, station_load_car, station_wait_for_car e station_on_board, usando apenas um mutex e variáveis de condição por estação, sem semáforos e sem busy-wait. Entrego o sync_car.c (com o sync_car.h), um Makefile que gera o executável sync_car, testes extras de casos limite e inválidos (test_extra.c) e o README.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Definir a struct station e a station_init	OK	sync_car.h / sync_car.c; compila sem warnings no meu código
station_load_car só retorna com todos sentados e vagão cheio ou plataforma vazia	OK	./sync_car: "departed ... (expected N)" sem *****; T4 do make test
station_load_car retorna na hora sem passageiros ou sem assentos	OK	início do runner (load_car 0 e 10); T1
station_wait_for_car só retorna com carro na estação e assento livre	OK	./sync_car; T2 (carro com assentos negativos: ninguém embarca)
station_on_board avisa o carro que o passageiro embarcou	OK	./sync_car; T3 (carro só parte após o último on_board)
Usar só pthread_mutex_* e pthread_cond_* (sem semáforos)	OK	grep -n "sem_" sync_car.c (sem resultado)
Um único lock por struct station	OK	struct station tem só o campo lock
Embarque simultâneo (vários retornos de wait_for_car antes de on_board)	OK	T3: 3 passageiros retornam antes de qualquer on_board e o carro espera
Sem busy-wait	OK	toda espera é pthread_cond_wait dentro de while
Compilação por Makefile gerando o executável sync_car	OK	make clean && make; ls sync_car

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
make test
for i in $(seq 1 50); do ./sync_car > /dev/null || echo "falhou $i"; done
```

./sync_car roda o teste do professor (deve terminar com "Looks good!"). make test roda o teste do professor e os testes extras (deve terminar com "Todos os testes extras passaram!"). O laço roda o teste 50 vezes; se não imprimir nada, passou em todas. Ambiente: Ubuntu no WSL (Windows), gcc.

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.h → define a struct station (1 mutex, 2 variáveis de condição, 3 contadores) e os protótipos**

2. **sync_car.c → implementa station_init, station_load_car, station_wait_for_car e station_on_board**
3. **test_extra.c → testes de casos inválidos (assentos negativos) e limite (embarque simultâneo, sobra de assentos)**
4. **Makefile → compila sync_car e test_extra; make test roda os testes; make clean limpa**

Decisões técnicas:

1. **Contador boarding (assento reservado, ainda sem on_board) → o enunciado permite vários retornos de wait_for_car antes de on_board → o carro só parte com boarding == 0, nunca com alguém na porta**
2. **Duas variáveis de condição: car_arrived (passageiros) e car_loaded (carro) → cada lado espera um evento diferente → broadcast acorda todos os passageiros quando o carro chega; signal acorda só o carro**
3. **Reservar o assento dentro de wait_for_car (free_seats--, com o lock) e zerar free_seats quando o carro parte → evita dois passageiros no mesmo assento e embarque em carro que já saiu → assentos 0 ou negativos se comportam como "carro sem lugar"**
4. **pthread_cond_wait sempre dentro de while → spurious wakeups e corrida por assento → a thread confere a condição de novo ao acordar**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

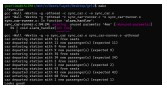
5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex + 2 condições + 3 contadores	Um lock por estação; passageiros dormem em car_arrived, o carro dorme em car_loaded; waiting, free_seats e boarding decidem quem pode seguir	Não houve troca: a primeira versão passou em todos os testes
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Gerar a implementação (struct + 4 funções) com IA e revisar a lógica	OK		03/11 ~07:10	sync_car.c; seção 7
2	S1	Rodar sed no Makefile a partir do Desktop	Falha	Estava fora da pasta grl2, então o Makefile não foi achado	03/11 07:20	"sed: can't read Makefile"
3	S1	mv Makefile.txt Makefile	Falha	Desnecessário: o arquivo já se chamava Makefile	03/11 07:20	
4	S1	make dentro de grl2 (WSL)	Parcial: compilou com 1 warning	Warning só no sync_car-runner.c (parâmetro foo não usado), não no meu código	03/11 07:20	saída do make 

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
5	S1	make test (1ª tentativa)	Falha	Makefile da pasta era o antigo, sem a regra test, e faltava o test_extra.c	03/11 19:16	evidencias.log ; print 1
6	S1	Recriar Makefile e test_extra.c pelo terminal (cat << EOF)	OK		03/11 19:17	evidencias.log
7	S1	make test (runner + testes extras T1–T4)	OK: "Looks good!" e 4/4 testes extras		03/11 19:18	evidencias.log ; print 2
8	S1					

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

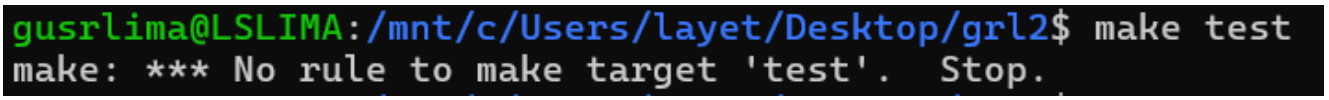
Sessão registrada em evidencias.log (dentro do grl2.tar): primeira tentativa de make test (falha), recriação do Makefile e do test_extra.c e make test com sucesso. Sessão: 03/11/2026, das 19:14:34 às 19:19:24 ("Script started" / "Script done" no log).

6.2 Prints

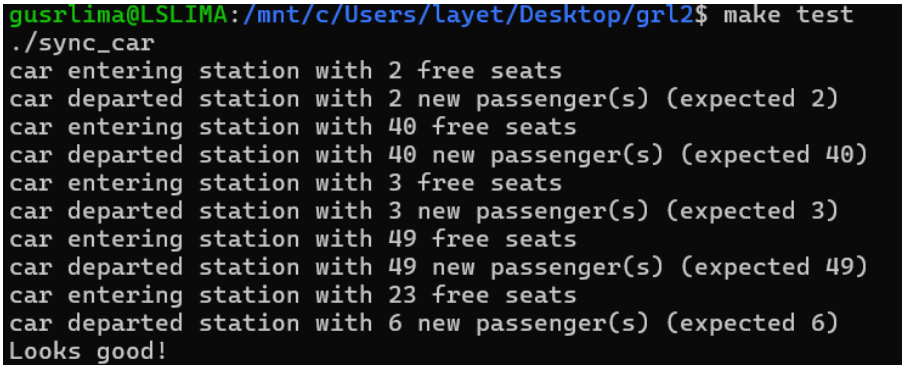
- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

Print 1 – Erro principal: terminal com "make: *** No rule to make target 'test'. Stop." (o Makefile da pasta ainda era o antigo).



Print 2 – Validação final: saída do make test terminando em "Looks good!" e "Todos os testes extras passaram!".



7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

- Compilei e rodei no meu WSL (make e make test, com o runner e os testes extras), registrado no evidencias.log das 19:14 às 19:19
- Conferi no sync_car.c que há um único mutex e só as funções pthread permitidas (sem semáforos)
- Conferi que toda espera usa pthread_cond_wait dentro de while (sem busy-wait)
- Acompanhei um exemplo à mão (3 passageiros, carro com 2 lugares), contador por contador, para entender a lógica.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A decisão técnica central foi o contador boarding: sem ele, o carro poderia partir assim que os assentos acabassem, com passageiros que já saíram de wait_for_car e ainda não chamaram on_board. Também foi importante zerar free_seats quando o carro parte, para ninguém embarcar num carro que já saiu. Os erros que tive foram de ambiente, não de lógica: rodei comandos fora da pasta do projeto, tentei renomear um arquivo que já tinha o nome certo e rodei make test com o Makefile antigo. Na próxima vez, começaria o script -a evidencias.log desde o primeiro comando, para o diário ficar todo comprovado no log. Como a implementação foi gerada com ajuda de IA, com mais tempo eu escreveria uma versão própria antes de comparar e faria mais testes de estresse para ganhar confiança na sincronização.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim (make no WSL/Ubuntu)
Rodei os testes e evidencias.log foi gerado	Sim (evidencias.log, 19:14–19:19)
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim (make test: T1/T2 inválidos, T3/T4 limite)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

- Robustez: tratar count negativo explicitamente como 0 no station_load_car, em vez de depender do comportamento implícito (hoje coberto pelos testes T1 e T2).
- Robustez: conferir o retorno de pthread_mutex_lock/pthread_cond_wait e criar uma station_destroy com pthread_mutex_destroy e pthread_cond_destroy.
- Testes: compilar com -fsanitize=thread e rodar valgrind --tool=helgrind para detectar data races automaticamente.
- Testes: estresse com vários carros seguidos e 1.000+ passageiros, rodando o laço 500 vezes.
- Warnings: compilar meu código com -Werror para nenhum warning novo passar despercebido.
- Organização: um script test.sh que roda tudo N vezes e grava o resumo no evidencias.log.

Regras de Integridade

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.